

Optimizing Dynamic Graph CNNs for Edge Deployment

From A100 Training to Jetson Nano Inference

Kinshuk Gupta

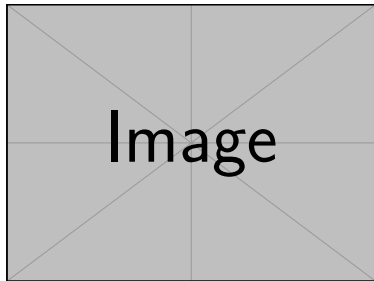
May 7, 2026

Outline

- 1 Introduction to DGCNN
- 2 Dataset & Objective
- 3 Baseline Implementation
- 4 Optimizations & Experiments
- 5 Overall Results & Visualizations
- 6 Conclusions

What is a Point Cloud?

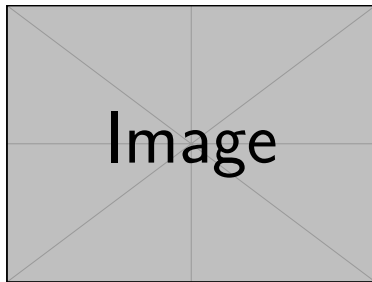
- A point cloud is a set of data points in space, usually representing a 3D shape or object.
- Irregular format: Unlike images (grid of pixels), point clouds are unordered sets of points (x, y, z) .
- Standard CNNs cannot be applied directly because they expect data on a regular grid.



Placeholder: Insert a point cloud image here

Dynamic Graph CNN (DGCNN) - Intuition

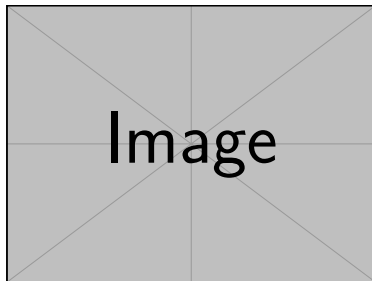
- **EdgeConv:** The core building block. Constructs a local neighborhood graph (k-NN) and extracts edge features between a point and neighbors.
- **Dynamic Graph Construction:** After each layer, features are updated, and the k-NN graph is recomputed in the new feature space.
- **Result:** Learns both local geometric features and global shape properties dynamically.



Placeholder: Insert DGCNN architecture diagram here

Dataset: ModelNet10

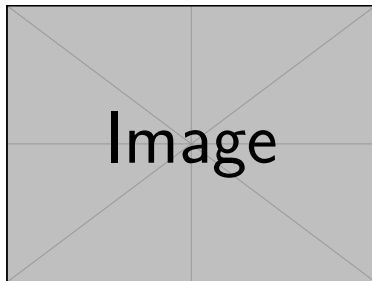
- 10 categories of CAD models (e.g., bathtub, bed, chair).
- We sample 1024 points per object from the mesh surfaces.
- **Preprocessing:** Center and normalize point clouds to a unit sphere.
- **Data Augmentation:** Random rotation, jitter, and scaling applied during training.



Placeholder: Insert ModelNet10 examples here

The Objective: Edge Deployment

- **Target Device:** NVIDIA Jetson Nano (Maxwell GPU, 4GB RAM).
- **Challenge:** DGCNN is computationally expensive. Computing dynamic k-NN graphs and large MLPs in every layer is slow and memory-intensive.
- **Goal:** Optimize the architecture to reduce latency and memory footprint while maintaining high classification accuracy.

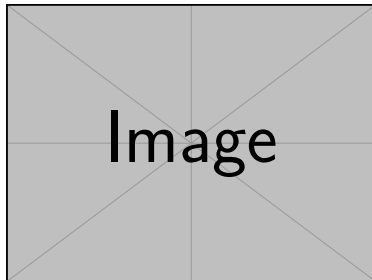


Placeholder: Insert Jetson Nano image here

Baseline Pipeline Overview

Our baseline implementation processes data in the following pipeline:

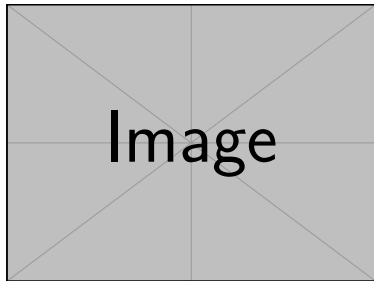
- 1 Point Cloud Input
- 2 KNN Graph Construction
- 3 EdgeConv Layer 1
- 4 EdgeConv Layer 2
- 5 EdgeConv Layer 3 (and 4)
- 6 Feature Concatenation
- 7 Global Pooling
- 8 Classifier → Object Class



TODO: Insert overall pipeline diagram

Step 1: Point Cloud Input

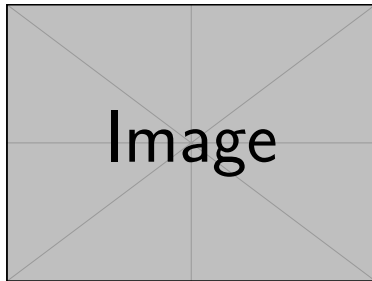
- **Input:** A batch of raw 3D coordinates.
- **Shape:** $N \times 3$, where $N = 1024$ points sampled from the object's surface.
- **Preprocessing:** The points are centered at the origin and normalized to fit within a unit sphere.



TODO: Insert normalized point cloud image

Step 2: KNN Graph Construction

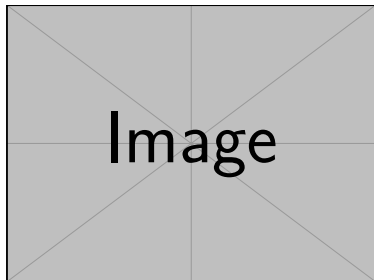
- **Operation:** For each point, we find its k nearest neighbors.
- **Baseline Value:** $k = 20$.
- **Distance Metric:** Euclidean distance in the current feature space (initially 3D coordinate space).
- **Result:** A directed graph connecting each point to its closest neighbors, capturing local topology.



TODO: Insert KNN graph diagram

Step 3: EdgeConv Layer 1

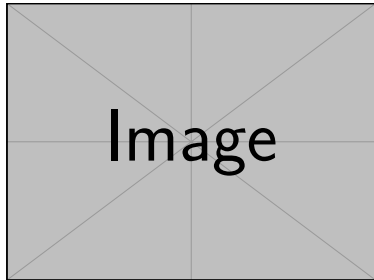
- **Input Features:** 3D coordinates (x, y, z) .
- **Edge Features:** Computes $(x_i, x_j - x_i)$ to capture both global position and local relative geometry.
- **MLP:** Passes these edge features through a Multi-Layer Perceptron.
- **Output:** 64-dimensional features. Aggregates neighborhood info via max pooling.



TODO: Insert EdgeConv Layer 1 feature visualization

Step 4: EdgeConv Layer 2

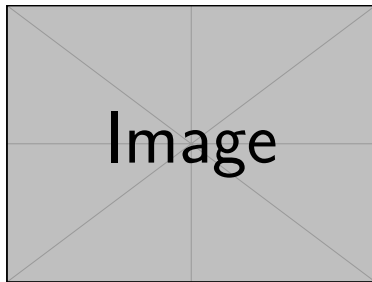
- **Dynamic Update:** Graph is recomputed! We find 20 nearest neighbors in the new 64-dimensional feature space, not just 3D space.
- **Semantic Grouping:** Points that are semantically similar (e.g., all parts of a chair leg) move closer together in feature space.
- **Output:** A new set of 64-dimensional features.



TODO: Insert dynamic graph feature space image

Step 5: EdgeConv Layers 3 & 4

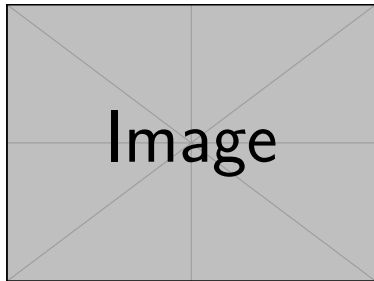
- **Deep Features:** The process repeats, expanding the receptive field.
- **Dimensions:** Layer 3 outputs 128 channels, Layer 4 outputs 256 channels.
- **Purpose:** Captures highly abstract, global topological structures of the 3D object.



TODO: Insert deep feature abstract representation

Step 6: Feature Concatenation

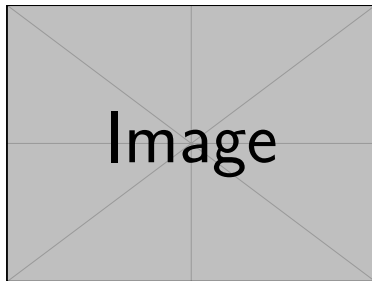
- **Multi-Scale Fusion:** We concatenate the outputs of all EdgeConv layers: $64 + 64 + 128 + 256 = 512$ channels.
- **Projection:** A 1D convolutional layer processes this concatenated tensor.
- **Embedding:** Maps the 512 channels to a high-dimensional space of 1024 dimensions.



TODO: Insert concatenation block diagram

Step 7: Global Pooling

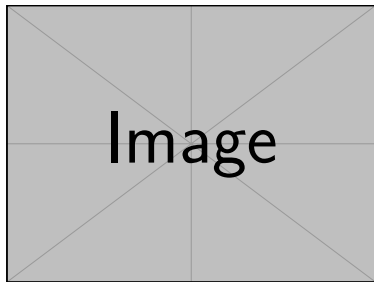
- **Problem:** Point clouds are unordered. We need a permutation-invariant descriptor.
- **Symmetric Function:** We apply Global Max Pooling and Global Mean Pooling across all N points.
- **Result:** The $(N \times 1024)$ tensor collapses into a single (1×2048) 1D global shape vector.



TODO: Insert max/mean pooling visualization

Step 8: Classifier & Object Class

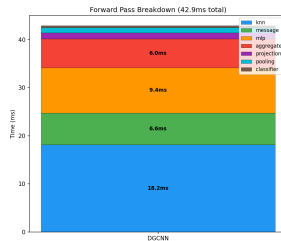
- **Classification Head:** A 3-layer MLP processes the 2048-dimensional shape vector.
- **Regularization:** Uses BatchNorm and Dropout (50%) to prevent overfitting.
- **Output:** Logits for the 10 ModelNet categories.
- **Final Prediction:** The class with the highest probability (e.g., "Chair").



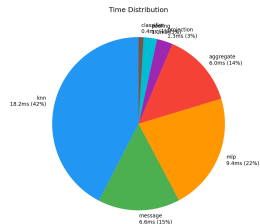
TODO: Insert classifier output probabilities diagram

Baseline Performance Summary

- **Test Accuracy:** 76.65%.
- **Latency:** ~ 42.4 ms/batch.
- **Size:** ~ 1.8 Million parameters.
- **Bottlenecks:** Profiling reveals that k-NN graph construction and MLP computations consume the majority of inference time.

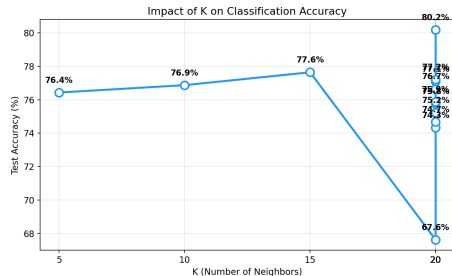


Baseline FP32 Latency Breakdown



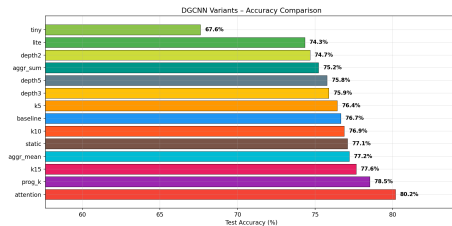
Optimization 1: K-Neighbors Sweep

- **Why?** The k-NN computation and edge feature processing scale linearly with k . Reducing k directly reduces computation.
- **Experiment:** Tested $k = 5, 10, 15, 20$.
- **Finding:** Increasing k beyond a certain point doesn't always yield better accuracy but incurs a strict latency penalty.



Optimization 2: Progressive K

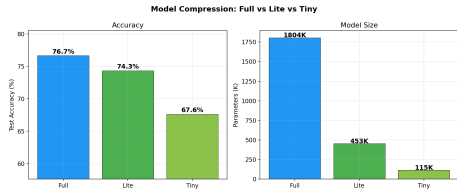
- **Why?** In deeper layers, the receptive field is already large. We don't need to connect to as many neighbors.
- **How?** Decrease k in each successive EdgeConv layer (e.g., $k = [20, 15, 10, 5]$).
- **Result:** Achieved **78.52%** accuracy (better than baseline!) while reducing latency to ~ 29.6 ms.



Progressive K ranks high in accuracy

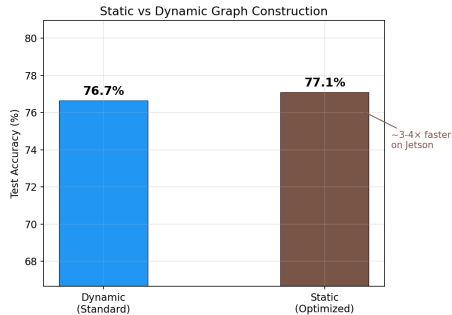
Optimization 3: Model Compression

- **Why?** 1.8M parameters is large for Jetson Nano's memory bandwidth.
- **How?** Reduce the number of channels in EdgeConv MLPs.
- **Lite:** Channels [32, 32, 64, 128], Emb: 512 (~450K params).
- **Tiny:** Channels [16, 16, 32, 64], Emb: 256 (~114K params).
- **Result:** Significant parameter reduction. Tiny drops to 67.6% acc but runs exceptionally fast at 24 ms/batch.



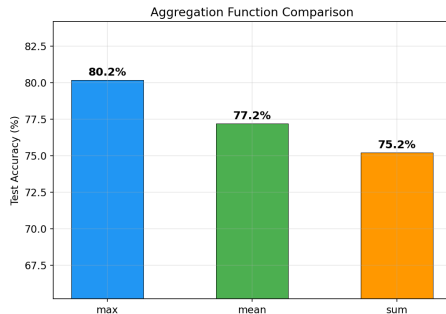
Optimization 4: Static vs Dynamic Graph

- **Why?** Recomputing the k-NN graph in feature space at every layer takes a huge portion of the total inference time.
- **How?** Compute the k-NN graph once on the raw (x, y, z) coordinates and reuse these edges statically.
- **Result:** Latency drops from ~ 42 ms to ~ 28 ms. Accuracy actually **improved** slightly (77.09%).



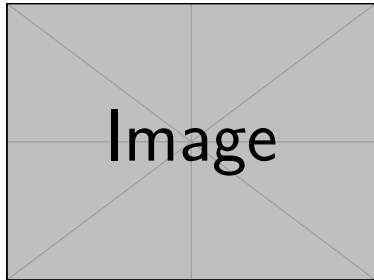
Optimization 5: Aggregation Functions

- **Why?** The original DGCNN uses max pooling. Is it the optimal choice?
- **Experiment:** Compared max, mean, and sum aggregation.
- **Result:** mean aggregation performed slightly better than max in this setup (77.2%), while sum lagged behind.



Optimization 6: Edge Attention Gating

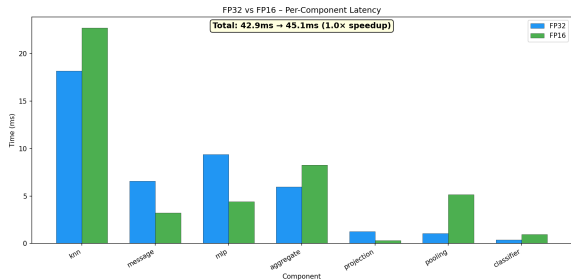
- **Why?** Not all neighbors are equally important.
- **How?** Add a small attention MLP that outputs a sigmoid scalar for each edge, multiplying the edge features to learn which neighbors matter.
- **Result:** Achieved the **highest accuracy (80.18%)** but at the cost of higher latency (~ 46.5 ms).



Placeholder: Insert Attention Mechanism diagram

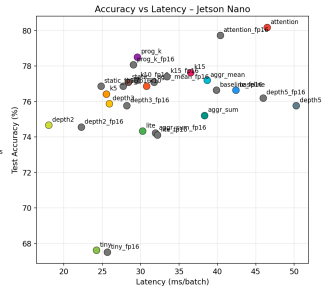
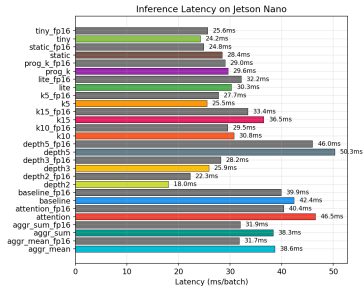
Optimization 7: FP16 Inference

- **Why?** Jetson Nano's GPU supports half-precision (FP16), speeding up memory-bound operations.
- **How?** Cast model and inputs to `torch.float16`.
- **Result:** MLP and Projection layers see the biggest speedups, resulting in an overall 10-15% latency reduction across all variants.



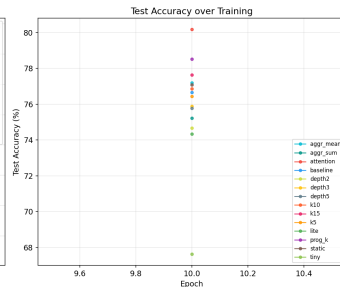
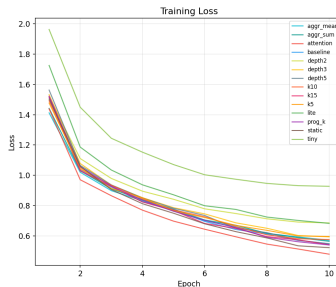
Accuracy vs Latency (Pareto Front)

- The tradeoff space across all 14 variants.
- Progressive K and Static Graph offer the best Pareto-optimal points for the edge.



Training Stability

- Training curves across all 14 variants demonstrate stable convergence.
- No signs of catastrophic forgetting or mode collapse.



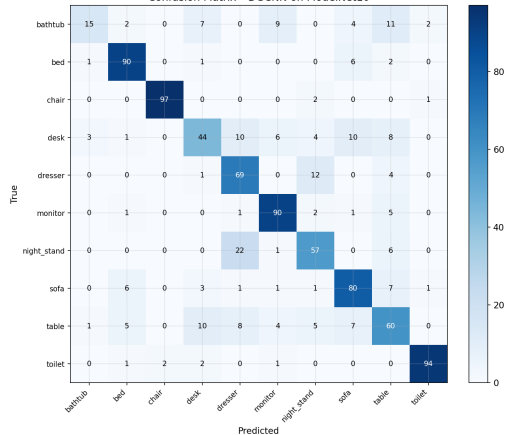
Qualitative Results: Features and Confusion Matrix

t-SNE of Learned Features (DGCNN)



t-SNE of Learned Features

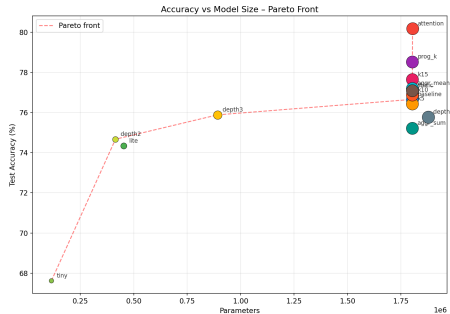
Confusion Matrix - DGCNN on ModelNet10



ModelNet10 Confusion Matrix

Conclusions & Key Takeaways

- **Highest Accuracy:** Edge Attention (80.18%).
- **Fastest Inference:** Tiny variant (24 ms/batch), but significant accuracy drop (67.6%).
- **Best Trade-offs (Pareto Optimal):**
 - **Progressive K:** 78.52% acc @ 29.6 ms.
 - **Static Graph:** 77.09% acc @ 28.4 ms.
- **FP16:** 10-15% latency reduction.
- **Recommendation:** Use **Progressive K** or **Static Graph** with **FP16** enabled.



Thank You